

EXP 7 : Generative Models with GANs

Aim

To construct and train a **Generative Adversarial Network (GAN)** using TensorFlow/Keras on the MNIST dataset to generate synthetic handwritten digit images.

Algorithm

1. Load the MNIST dataset.
2. Normalize images to the range $[-1, 1]$.
3. Create a Generator using Dense and Conv2DTranspose layers.
4. Create a Discriminator using Conv2D and Dense layers.
5. Define Generator and Discriminator loss functions.
6. Define Adam optimizers.
7. Generate fake images from random noise.
8. Train the Discriminator using real and fake images.
9. Train the Generator to fool the Discriminator.
10. Repeat the process for multiple epochs.
11. Generate and display synthetic images.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
import os
(x_train, _), (_, _) = mnist.load_data()
x_train = x_train.reshape(x_train.shape[0], 28, 28, 1).astype("float32")
x_train = (x_train - 127.5) / 127.5
latent_dim = 100
batch_size = 256
generator = keras.Sequential([
    layers.Dense(7 * 7 * 256, use_bias=False, input_shape=(latent_dim,)),
    layers.BatchNormalization(),
```

```

layers.LeakyReLU(),
layers.Reshape((7, 7, 256)),
layers.Conv2DTranspose(128, (5, 5), padding="same", use_bias=False),
layers.BatchNormalization(),
layers.LeakyReLU(),
layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding="same", use_bias=False),
layers.BatchNormalization(),
layers.LeakyReLU(),
layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding="same", use_bias=False,
activation="tanh")
])
discriminator = keras.Sequential([
    layers.Conv2D(64, (5, 5), strides=(2, 2), padding="same", input_shape=(28, 28, 1)),
    layers.LeakyReLU(),
    layers.Dropout(0.3),
    layers.Conv2D(128, (5, 5), strides=(2, 2), padding="same"),
    layers.LeakyReLU(),
    layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(1, activation="sigmoid")
])
cross_entropy = keras.losses.BinaryCrossentropy()
def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)
def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss
generator_optimizer = keras.optimizers.Adam(learning_rate=1e-4)
discriminator_optimizer = keras.optimizers.Adam(learning_rate=1e-4)
@tf.function
def train_step(images):
    noise = tf.random.normal([batch_size, latent_dim])
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)
        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)
        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

```

```

gen_gradients = gen_tape.gradient(
    gen_loss, generator.trainable_variables
)
disc_gradients = disc_tape.gradient(
    disc_loss, discriminator.trainable_variables
)
generator_optimizer.apply_gradients(
    zip(gen_gradients, generator.trainable_variables)
)
discriminator_optimizer.apply_gradients(
    zip(disc_gradients, discriminator.trainable_variables)
)
return gen_loss, disc_loss
train_dataset = tf.data.Dataset.from_tensor_slices(x_train)
train_dataset = train_dataset.shuffle(x_train.shape[0]).batch(batch_size)
seed = tf.random.normal([16, latent_dim])
def generate_images(epoch):
    predictions = generator(seed, training=False)
    predictions = (predictions * 0.5) + 0.5
    plt.figure(figsize=(4, 4))
    for i in range(16):
        plt.subplot(4, 4, i + 1)
        plt.imshow(predictions[i, :, :, 0], cmap="gray")
        plt.axis("off")
    plt.suptitle(f"Epoch {epoch}", fontsize=16)
    plt.tight_layout()
    plt.show()
EPOCHS = 20
for epoch in range(EPOCHS):
    gen_losses = []
    disc_losses = []
    for image_batch in train_dataset:
        gen_loss, disc_loss = train_step(image_batch)
        gen_losses.append(gen_loss.numpy())
        disc_losses.append(disc_loss.numpy())
    avg_gen_loss = np.mean(gen_losses)
    avg_disc_loss = np.mean(disc_losses)
    print(
        f"Epoch {epoch + 1}/{EPOCHS} - "
        f"Generator Loss: {avg_gen_loss:.4f} - "

```

```

        f"Discriminator Loss: {avg_disc_loss:.4f}"
    )
    if (epoch + 1) % 5 == 0:
        generate_images(epoch + 1)
print("Training complete.")
generate_images(EPOCHS)

```

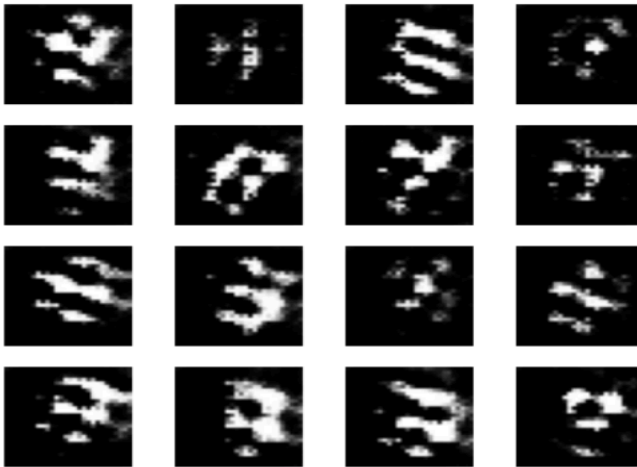
Output:

```

Epoch 1/20 - Generator Loss: 0.8968 - Discriminator Loss: 0.8676
Epoch 2/20 - Generator Loss: 0.8844 - Discriminator Loss: 1.1852
Epoch 3/20 - Generator Loss: 1.1953 - Discriminator Loss: 0.9760
Epoch 4/20 - Generator Loss: 1.0053 - Discriminator Loss: 1.1176
Epoch 5/20 - Generator Loss: 1.1004 - Discriminator Loss: 1.1658

```

Epoch 5

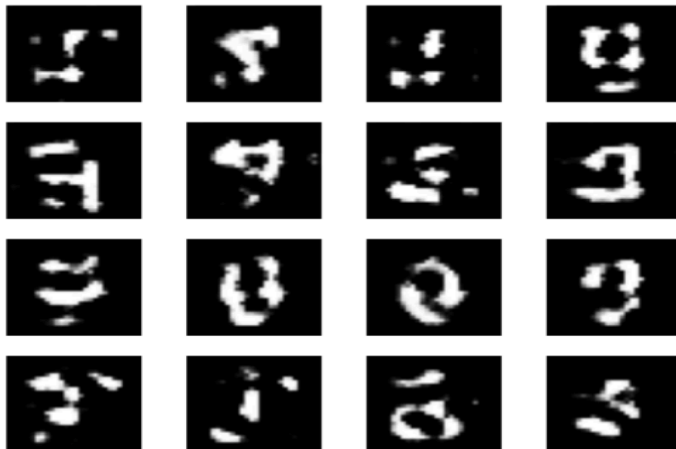


```

Epoch 6/20 - Generator Loss: 0.9122 - Discriminator Loss: 1.2580
Epoch 7/20 - Generator Loss: 0.9827 - Discriminator Loss: 1.1908
Epoch 8/20 - Generator Loss: 0.9356 - Discriminator Loss: 1.2244
Epoch 9/20 - Generator Loss: 0.8871 - Discriminator Loss: 1.2436
Epoch 10/20 - Generator Loss: 0.9811 - Discriminator Loss: 1.1522

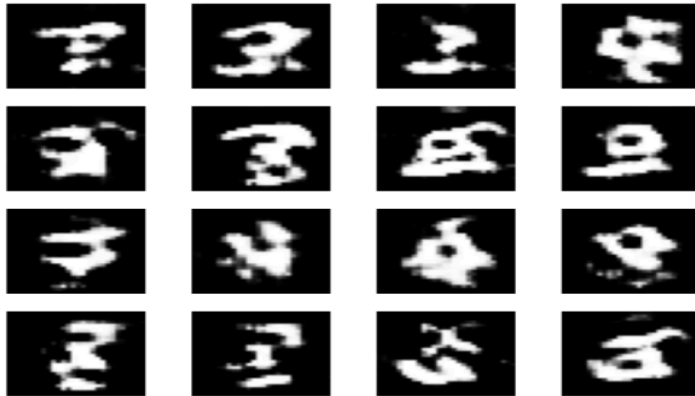
```

Epoch 10



Epoch 11/20 - Generator Loss: 0.9516 - Discriminator Loss: 1.2539
Epoch 12/20 - Generator Loss: 1.0930 - Discriminator Loss: 1.1373
Epoch 13/20 - Generator Loss: 0.9460 - Discriminator Loss: 1.2264
Epoch 14/20 - Generator Loss: 1.0655 - Discriminator Loss: 1.1781
Epoch 15/20 - Generator Loss: 1.0288 - Discriminator Loss: 1.1547

Epoch 15



Epoch 16/20 - Generator Loss: 1.1528 - Discriminator Loss: 1.0437
Epoch 17/20 - Generator Loss: 1.1194 - Discriminator Loss: 1.1148
Epoch 18/20 - Generator Loss: 1.1678 - Discriminator Loss: 1.0374
Epoch 19/20 - Generator Loss: 1.2792 - Discriminator Loss: 1.0021
Epoch 20/20 - Generator Loss: 1.2795 - Discriminator Loss: 0.9854

Epoch 20



Training complete.

Result:

GAN was successfully implemented and trained. The Generator produced synthetic handwritten digit images from MNIST data